

Tutorial paso a paso

Metodo final enviado al SGSMA Challenge

Physics-informed PMU event detection and localization

Modelo final: `sgms_extra_trees_windowed_v2`

Variante: `base_v2 + rolling + rls_kalman + graph_temporal`

Mapa de 30 minutos

Parte 1: problema y datos

- ▶ Que entra y que debe salir.
- ▶ Eventos 0–8.
- ▶ Por que usamos ventanas.

Parte 2: features fisicas

- ▶ Baseline pre-evento.
- ▶ Estadísticas, derivadas, z robusto.
- ▶ Calidad de datos, bad data y potencia proxy.

Parte 3: dinamica y ML

- ▶ Rolling, RLS/Kalman y graph temporal.
- ▶ ExtraTrees jerarquico.
- ▶ Reglas fisicas de proteccion.

Parte 4: localizacion y resultado

- ▶ Localizadores BUS, LINE y PMU.
- ▶ Pipeline final de inferencia.
- ▶ Resultados y receta para reprogramar.

Objetivo del challenge

Entrada

Series temporales PMU por bus observado:

- ▶ Voltajes y corrientes trifasicas.
- ▶ Angulos de tension y corriente.
- ▶ Frecuencia y ROCOF.
- ▶ Calidad de datos.

Salida requerida

Archivo CSV con:

- ▶ TIMESTAMP
- ▶ Bus
- ▶ Predicted_Event
- ▶ Predicted_Location

Tarea real

No basta detectar anomalia. Hay que decidir **que evento ocurrio** y **donde ocurrio**: bus, linea o PMU.

Eventos que predice el metodo

Label	Evento	Lectura fisica principal
0	normal	Sin perturbacion ni problema de datos.
1	fault	Cambio brusco de voltaje, alta derivada, sag/transitorio.
2	line outage	Redistribucion de corriente, sin caida extrema de voltaje.
3	generation change	Cambio de frecuencia, ROCOF y potencia activa proxy.
4	load change	Cambio sostenido de corriente/potencia consumida.
5	missing data	Datos faltantes sin evidencia fisica fuerte.
6	missing + physical	Missing simultaneo con evento fisico.
7	bad data	Una senal aislada se vuelve incoherente.
8	mixed proxy	Firma fisica mixta/no limpia.

Señales disponibles

Señales eléctricas

Grupo	Columnas
Voltaje mag.	VA_MAG, VB_MAG, VC_MAG
Corriente mag.	IA_MAG, IB_MAG, IC_MAG
Voltaje ang.	VA_ANG, VB_ANG, VC_ANG
Corriente ang.	IA_ANG, IB_ANG, IC_ANG
Dinámica	Freq, ROCOF

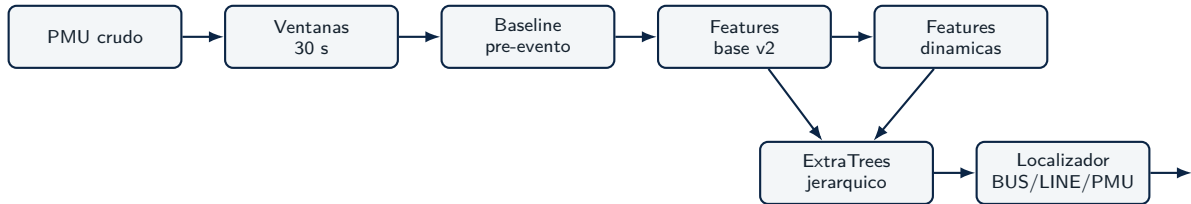
Señales de soporte

- ▶ `TIMESTAMP`
- ▶ `DATA_PRESENT`
- ▶ Nombre del archivo `Bus*.csv`
- ▶ Topología IEEE 39 para distancias eléctricas.

Idea clave

Cada columna se transforma en una evidencia física: severidad, rapidez, persistencia, calidad y propagación espacial.

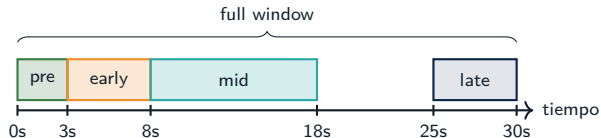
Pipeline final en una sola imagen



Principio del metodo

El modelo no memoriza la senal cruda. Primero convierte cada ventana en una firma fisica y despues usa ML robusto para decidir evento y localizacion.

Unidad de decision: ventana de 30 segundos



Por que 30 s

- ▶ Captura inicio, transitorio y estado posterior.
- ▶ Evita decidir con una sola muestra.
- ▶ Permite generar features estables.
- ▶ La salida se replica a timestamps de la ventana.

Lectura fisica

Fault tiende a aparecer en early; cambios de generacion/carga se ven mejor en mid/late; missing se ve como gaps a lo largo de la ventana.

Baseline pre-evento

Paso

Para cada PMU y cada senal, tomar la mediana de los primeros segundos:

$$b_x = \text{median}(x(t), t \in \text{pre})$$

Magnitudes

$$x_{\text{pu}}(t) = \frac{x(t)}{b_x + \epsilon}$$
$$\Delta x(t) = x(t) - b_x$$

Angulos

Primero se aplica unwrap para no confundir saltos de representacion con eventos reales.

$$\theta_{\text{clean}} = \text{unwrap}(\theta)$$

Por que importa

Cada PMU se compara contra su propio estado normal. Asi el modelo aprende cambios relativos, no valores absolutos fragiles.

Familias de features base v2

Familia	Que calcula	Que significa
Estadísticas por señal	media, std, min, max, span, percentiles	magnitud y severidad
Deltas temporales	early-pre, mid-pre, late-pre	evolución del evento
Derivadas	max derivada, RMS derivada	rapidez/transitorio
Robust z-score	desvío contra MAD pre-evento	anomalía relativa
Data quality	gaps, NaNs, DATA_PRESENT	missing o PMU caído
Single-signal	max/second/ratio de anomalías	bad data aislado
Power proxy	P, Q, ángulo de factor de potencia	generación/carga
Agregados globales	mean/max/min/std entre PMUs	patrón espacial

Features estadísticas por señal

Se calculan para cada señal y cada segmento

{mean, std, min, max, span, median, max_abs, mean_abs, p05, p95}

Ejemplo de nombre de feature

Bus2__VA_MAG__early__span

Bus10__Freq__late__mean

Interpretacion

- ▶ **span**: cuanto se movio la señal.
- ▶ **std**: oscilacion o inestabilidad.
- ▶ **p05/p95**: extremos robustos.
- ▶ **mean/median**: cambio de nivel.

Deltas temporales y derivadas

Deltas contra pre-evento

$$\Delta_{\text{early}} = \mu_{\text{early}} - \mu_{\text{pre}}$$

$$\Delta_{\text{mid}} = \mu_{\text{mid}} - \mu_{\text{pre}}$$

$$\Delta_{\text{late}} = \mu_{\text{late}} - \mu_{\text{pre}}$$

Derivadas

$$d_x(t) = \frac{x(t) - x(t - \Delta t)}{\Delta t}$$

max_abs_derivative, rms_derivative

Uso por evento

- ▶ Fault: derivada alta en voltaje.
- ▶ Line outage: derivada alta en corriente.
- ▶ Load/generation: late-pre sostenido.

Robust z-score

Definicion

$$\begin{aligned}m &= \text{median}(x_{\text{pre}}) \\ \text{MAD} &= \text{median}(|x_{\text{pre}} - m|) \\ z_{\text{robust}}(t) &= \frac{|x(t) - m|}{\text{MAD} + \epsilon}\end{aligned}$$

Features

- ▶ `max_abs_robust_z`
- ▶ `sustained_abs_robust_z_5`
- ▶ `sustained_abs_robust_z_15`

Interpretacion

Mide que tan rara fue una senal respecto a su propio estado pre-evento. Es mas estable que usar solo media y desviacion estandar.

Data quality: detectar missing

Features principales

- ▶ `n_samples`
- ▶ `dt_s, duration_s`
- ▶ `max_timestamp_gap_s`
- ▶ `max_timestamp_gap_ratio`
- ▶ `data_present_fraction`
- ▶ `data_present_min`
- ▶ `missing_run_max_s`
- ▶ `nan_fraction_max/mean`

Decision conceptual

$\text{missing} \Rightarrow \begin{cases} \text{Event 5,} & \text{si no hay evento fisico} \\ \text{Event 6,} & \text{si hay evento fisico} \end{cases}$

Senal fisica

Gaps temporales o DATA_PRESENT bajo indican falla de medicion/comunicacion, no necesariamente falla electrica.

Bad data: una senal aislada rompe el patron

Features single-signal

- ▶ `single_signal_score_max`
- ▶ `single_signal_score_second`
- ▶ `single_signal_score_ratio`
- ▶ `single_signal_score_count_gt20`

Intuicion

Si una sola columna explota y las demas siguen coherentes, probablemente no es un evento electrico global: es dato corrupto.

Regla mental

$$\frac{\text{score}_1}{\text{score}_2} \geq 3 \Rightarrow \text{bad data candidate}$$

Power proxy: aproximar P y Q sin potencia medida

Por fase

$$\phi = \theta_V - \theta_I$$

$$P_{\text{proxy}} = V_{\text{pu}} I_{\text{pu}} \cos(\phi)$$

$$Q_{\text{proxy}} = V_{\text{pu}} I_{\text{pu}} \sin(\phi)$$

$$PF_{\text{ang}} = \phi$$

Para que sirve

- ▶ Event 3: cambio de generacion.
- ▶ Event 4: cambio de carga.
- ▶ Event 8: firma fisica mixta.

Idea

Generacion/carga muchas veces se separan mejor por potencia y frecuencia que por voltaje.

Tamano de las features base

Grupo	Numero aproximado
Voltage angle	7,596
Voltage magnitude	7,596
Current angle	7,596
Current magnitude	7,596
Frequency / ROCOF	4,640
Power proxy	4,428
Data quality	156
Single-signal anomaly	48
Other/global	60
Total base v2	39,716

Lectura

El modelo no ve miles de puntos crudos; ve una firma compacta y físicamente interpretable de cada ventana.

Por que agregar features dinamicas

Limitacion de features base

Las estadísticas resumen **cuanto** cambio una señal, pero no siempre capturan bien **como** evoluciono ni **como se propago** entre PMUs.

Rolling

Escala temporal corta y larga.

RLS/Kalman

Residuo e innovación contra una dinámica suave.

Graph temporal

Diferencias entre PMUs y tiempo de llegada.

Variante final

`base_v2 + rolling + rls_kalman + graph_temporal`

Bloque rolling

Ventanas internas

$$w \in \{5, 15, 30, 60, 120\}$$

Se aplican sobre senales como:

- ▶ VA_MAG, IA_MAG
- ▶ VA_ANG, IA_ANG
- ▶ Freq, ROCOF

Features

- ▶ mean_abs_max
- ▶ std_max, std_mean
- ▶ slope_max_abs
- ▶ energy
- ▶ time_to_abs_peak

Significado

Rapidez, energia sostenida y momento del pico. Ayuda a distinguir transitorios bruscos de cambios lentos.

Bloque RLS/Kalman

Idea

Un filtro espera que la señal siga una trayectoria suave. Cuando ocurre un evento, el residuo o la innovación sube.

Señales usadas

VA_MAG, IA_MAG, VA_ANG, IA_ANG, Freq, ROCOF

Features

- ▶ level_resid
- ▶ theta_delta
- ▶ linear_resid
- ▶ kalman_innov
- ▶ max, mean, p95, late-pre, time-to-peak

Lectura física

Si el cambio no se explica por tendencia normal, es evidencia de evento dinámico real.

Bloque graph temporal

Que compara

- ▶ Spread entre PMUs.
- ▶ Diferencias por pares de PMUs.
- ▶ Correlaciones por pares.
- ▶ Diferencias en tiempo al pico.
- ▶ Diferencias ponderadas por distancia electrica.

Por que localiza

Un evento cerca de un bus o linea afecta mas fuerte y antes a ciertos PMUs. El patron espacial ayuda a inferir el origen.

Features clave

time_to_peak_range
pair_diff
pair_corr
weighted_graph_diff

Feature vector final

Base

$$n_{\text{base}} \approx 39,716$$

- ▶ Senales fisicas.
- ▶ Calidad de datos.
- ▶ Potencia proxy.
- ▶ Agregados globales.

Final localizer

$$n_{\text{final}} = 45,162$$

- ▶ base_v2
- ▶ rolling
- ▶ rls_kalman
- ▶ graph_temporal

Decision final

Se escogio esta variante porque logro el mejor RAW localization entre variantes productivas, manteniendo buen nivel en SIM.

Modelo ML: SimpleImputer + ExtraTrees

Pipeline de cada modelo

features → median imputer → ExtraTreesClassifier

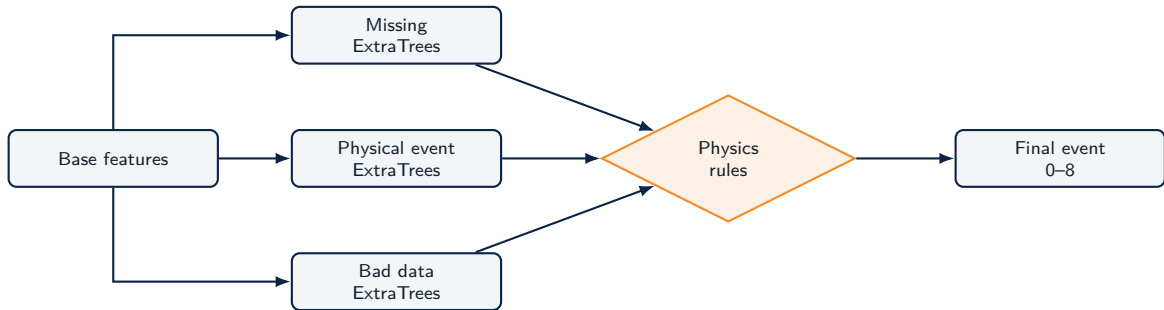
Por que imputar

Algunas ventanas pueden tener NaNs o columnas faltantes por problemas de datos. La mediana es simple y estable.

Por que ExtraTrees

- ▶ Maneja decenas de miles de features.
- ▶ No necesita escalado.
- ▶ Captura no-linealidad.
- ▶ Es robusto a outliers.
- ▶ Entrena rapido y sirve con datos mixtos.

Arquitectura jerarquica



No usamos un unico clasificador plano

Primero se predicen eventos fisicos principales; despues missing y bad data corrigen o reemplazan la decision usando reglas fisicas.

Modelos finales entrenados

Artefacto	Rol	Clases/Salida
physical_event_classifier	Clasificador fisico principal	0, 1, 2, 3, 4, 8
bad_data_detector	Detecta Event 7	bad/no bad
missing_composition_detector	Decide missing puro vs compuesto	missing/no missing
final_dynamic_localizers	Localizadores tipados	BUS, LINE, PMU

Ruta runtime

main.py → src/models/hybrid_submission.py → src/data_factory/final_model.py

Reglas físicas que protegen el clasificador

Fault-like

$$V_{\text{span}} \geq 50000$$
$$\max|dV/dt| \geq 500000$$

Line-outage-like

$$I_{\text{span}} \geq 500$$
$$\max|dI/dt| \geq 3000$$
$$V_{\text{span}} \leq 50000$$

Missing-like

$\text{DATA_PRESENT} < 0.999$
o NaNs/gaps altos.

Bad-data-like

$\text{single_signal_score}$ alto
 $\text{single_signal_ratio} \geq 3$

Logica final de evento

Orden conceptual

- 1 Predecir evento fisico con ExtraTrees: 0,1,2,3,4,8.
- 2 Medir si hay missing.
- 3 Medir si hay bad data.
- 4 Aplicar reglas de fault/line/missing/bad.
- 5 Emitir evento final 0-8.

Traduccion practica

- ▶ Missing sin fisica fuerte → Event 5.
- ▶ Missing con fisica fuerte → Event 6.
- ▶ Una senal aislada absurda → Event 7.
- ▶ Voltaje brutal → Event 1.
- ▶ Corriente fuerte sin sag brutal → Event 2.

Routing evento → localizador

Evento	Localizador
1 fault	BUS
2 line outage	LINE
3 generation	BUS
4 load	BUS
5 missing	PMU
6 missing + physical	BUS o LINE
7 bad data	PMU
8 mixed proxy	BUS

Por que tipado

No se localiza igual una linea abierta que un PMU con datos malos. Cada tipo de evento tiene geometria y firma fisica diferente.

Localizadores finales

BUS, BUS:event1, BUS:event3, BUS:event4, BUS:event8, LINE, LINE:event2, PMU

Como se localiza fisicamente

Evidencias que usa el localizador

- ▶ Que PMU tuvo mayor severidad.
- ▶ Que PMU vio primero el pico.
- ▶ Que pares de PMUs se movieron juntos.
- ▶ Como cambia corriente y angulo entre PMUs.
- ▶ Que firma encaja con distancia electrica a buses/lineas.

BUS

Severidad cercana a un nodo.

LINE

Redistribucion de corriente entre extremos/rutas.

PMU

Problema concentrado en el sensor.

Pipeline de inferencia final

```
case = read_pmu_case(input_dir)
windows = split_into_windows(case, seconds=30)

for window in windows:
    x_base = extract_window_features_v2(window)
    x_dyn = extract_dynamic_features(
        window,
        blocks=["rolling", "rls_kalman", "graph_temporal"],
    )

    physical = physical_model.predict(x_base)
    bad = bad_model.predict_proba(x_base)
    missing = missing_model.predict_proba(x_base)

    event = apply_hierarchical_physics_rules(
        physical, bad, missing, x_base
    )

    loc_type = route_event_to_localizer(event, physical)
    x_loc = align_features(concat(x_base, x_dyn))
    location = localizers[loc_type].predict(x_loc)

    write_same_prediction_for_all_rows(window, event, location)
```

Dataset y validacion

Entrenamiento

- ▶ Escenarios simulados SGSMA.
- ▶ Features base: `factory_features_v2.csv`.
- ▶ Features dinamicas: `dynamic_features_v3.csv`.
- ▶ Split SIM: 30% holdout estratificado.

Transferencia RAW

- ▶ Validacion en RAW0001.
- ▶ No usar etiquetas RAW para entrenar el modelo final.
- ▶ Medir detector, clasificador y localizador.

Regla de competencia

El modelo final debe generalizar a un dataset nuevo. Por eso se evitan variantes que “aprenden” RAW usando sus labels reales.

Resultado final

SIM holdout

Metric	Score
Detector accuracy	0.9693
Classifier accuracy	0.9673
Classifier macro-F1	0.9598
Localizer Top-1	0.8524

RAW0001

Metric	Score
Detector accuracy	1.0000
Classifier accuracy	1.0000
Classifier macro-F1	1.0000
Localizer Top-1	0.6667

Seleccion

La variante final fue elegida por equilibrio: rendimiento fuerte en SIM y mejor localizacion RAW entre variantes productivas.

Por que no fue DeepSeek el metodo final

Que paso

Se probaron experimentos DeepSeek y variantes exploratorias. Algunas daban numeros RAW muy altos.

Problema

Las mejores variantes DeepSeek usaban informacion de RAW con etiquetas reales para entrenar/ponderar. Eso mejora RAW0001, pero no es una configuracion valida para un dataset nuevo.

Conclusion correcta

DeepSeek fue util para explorar patrones, pero el envio final confiable fue SGMS con ExtraTrees jerarquico, features fisicas y localizadores dinamicos.

Implementacion desde cero: esqueleto

```
def solve_case(input_dir):  
    pmus = load_bus_csvs(input_dir)  
    windows = make_fixed_windows(pmus, seconds=30)  
  
    predictions = []  
    for win in windows:  
        base = extract_base_v2(win)  
        dyn = extract_dynamic_v3(  
            win,  
            include_blocks=("rolling", "rls_kalman", "graph_temporal"),  
        )  
        event, physical = predict_event_hierarchical(base)  
        location = predict_location(event, physical, base, dyn)  
        predictions.extend(expand_to_rows(win, event, location))  
  
    return make_submission_csv(predictions)
```

Mensaje para la audiencia

Si entienden estas cinco funciones, entienden todo el metodo final.

Receta minima si toca reprogramar en vivo

- 1 Leer todos los Bus*.csv.
- 2 Crear ventanas de 30 s.
- 3 Calcular baseline con la mediana de pre.
- 4 Extraer estadísticas, deltas, derivadas y z robusto.
- 5 Agregar calidad de datos y single-signal score.
- 6 Construir potencia proxy P/Q/PF.
- 7 Entrenar ExtraTrees para evento físico.
- 8 Entrenar detectores de missing y bad data.
- 9 Aplicar reglas jerárquicas.
- 10 Entrenar localizadores BUS, LINE y PMU.
- 11 Agregar rolling y graph temporal.
- 12 Agregar RLS/Kalman si queda tiempo.

Como explicar cada evento en la defensa

Evento	Features clave	Frase para defender
Fault	V span, dV/dt, z robusto	"Falla produce perturbacion fuerte de voltaje".
Line outage Generation	I span, dl/dt, graph features Freq, ROCOF, P proxy	"Linea fuera redistribuye corrientes". "Generacion cambia balance potencia-frecuencia".
Load Missing	I mag, P/Q proxy, late delta DATA_PRESENT, gaps, NaNs	"Carga cambia consumo sostenido". "Esto es problema de datos, no necesariamente fisico".
Bad data	single-signal ratio, z extremo	"Una senal aislada rompe coherencia".

Checklist final del envío

Archivos de runtime

- ▶ `main.py`
- ▶ `src/models/hybrid_submission.py`
- ▶ `src/data_factory/final_model.py`
- ▶ `models/final_model_config.json`
- ▶ `models/localizer/final_dynamic_localizers.joblib`

Ruta activa

- ▶ `model = sgms_extra_trees_windowed_v2`
- ▶ `route = ml_windowed`
- ▶ `window_seconds = 30`
- ▶ `salida: TIMESTAMP, Bus, Predicted_Event, Predicted_Location`

La idea en una frase

El metodo final convierte senales PMU crudas en features fisicas interpretables y usa ExtraTrees jerarquicos para detectar, clasificar y localizar eventos.

Fisica

Voltaje, corriente, angulo, frecuencia, calidad y potencia proxy.

ML

ExtraTrees robusto, imputacion mediana y reglas fisicas.

Localizacion

Firmas espaciales y temporales entre PMUs.

Receta final: `base_v2 + rolling + rls_kalman + graph_temporal`

Backup: umbrales físicos usados

Parametro	Valor
bad data probability	0.4211785821
missing composition probability	0.0
missing physical confidence	0.0
fault voltage span min	50000
fault voltage derivative min	500000
line current span min	500
line current derivative min	3000
line voltage span max	50000
bad single-signal score min	200000
bad single-signal ratio min	3.0
line gate max physical confidence	0.75
fault guard max physical confidence	0.60

Backup: artefactos finales

Bundle runtime

- ▶ `models/feature_columns.json`
- ▶ `models/classifiers/physical_event_classifier.joblib`
- ▶ `models/detector/bad_data_detector.joblib`
- ▶ `models/detector/missing_composition_detector.joblib`
- ▶ `models/detector/hierarchical_model_config.json`
- ▶ `models/localizer/final_dynamic_localizers.joblib`

Nota

El localizador hibrido grande redundante se excluyo del zip compacto; se reconstruye desde el bundle dinamico.

Backup: receta Event 1 fault

Detector manual

- 1 Tomar ventana de 30 s.
- 2 Calcular baseline de voltajes por PMU.
- 3 Medir V_{MAG} span, dV/dt , z robusto.
- 4 Si hay span o derivada extrema, marcar fault-like.
- 5 Rechazar si parece missing o bad data aislado.

Localizacion

Buscar el bus cuya firma espacial explica mejor donde fue mayor y mas temprano el sag/transitorio de voltaje.

Backup: receta Event 2 line outage

Detector manual

- 1 Medir cambios en corrientes y angulos.
- 2 Requerir corriente fuerte: span y derivada altos.
- 3 Verificar que voltaje no tenga sag brutal tipo fault.
- 4 Usar diferencias entre PMUs para ver redistribucion.

Localizacion

Comparar patron de corriente entre PMUs contra candidatos de linea. Una linea fuera cambia flujos, no solo una senal.

Backup: comandos de validacion

Ejecutar inferencia

```
python main.py -input-dir <RAW_OR_SIM_FOLDER> -output <OUTPUT_CSV>
```

Prueba SIM

```
python main.py -input-dir workbench\simulated\sgsma_generated\SIM00642\pmu -output  
workbench\_tmp\SIM00642_predictions.csv
```

Prueba RAW0001

```
python main.py -input-dir data\RAW0001 -output data\RAW0001\predictions.csv
```

Tests locales reportados

246 passed, 10 skipped

Backup: que decir si preguntan por generalizacion

Respuesta

La transferencia a un dataset nuevo depende de no aprender etiquetas RAW especificas. Por eso el modelo final usa simulacion, features fisicas, reglas y validacion RAW solo como evaluacion.

Riesgo principal

La localizacion es mas dificil que la deteccion/clasificacion porque pequenas diferencias topologicas o de PMU placement cambian la firma espacial.

Mejora futura

Agregar guardrails topologicos productivos y calibrar localizadores con el nuevo dataset sin usar etiquetas de prueba.